Where the NVARC Pipeline Saturates on ARC-AGI-2: Candidate-Pool Ceiling, Selection Headroom and Generation Collapse

Hack2Publish team*

Abstract

The pipeline that won ARC Prize 2025 (NVARC) is public, and our team built 34 ARC Prize 2026 notebooks on it. It combines a 4B-parameter Qwen3-based grid model, per-task LoRA test-time training, a probability-thresholded depth-first search over 16 augmented views of each test input, augmentation-based rescoring and two submitted attempts. We ask where this pipeline stops improving on the public ARC-AGI-2 evaluation set (120 tasks, 172 test inputs). We re-score 41 distinct results of these notebooks and analyse two full runs whose logs report, for every test input, the rank of the answer among the decoded candidates. Three measurements locate the limit. (i) *Candidate-pool ceiling*: a correct grid is among the decoded candidates for only 55 and 56 of 172 test inputs; 117 and 116 (68.0 % and 67.4 %) never receive a correct candidate. (ii) *Selection headroom*: an oracle that submits the correct candidate whenever one exists would raise the task-normalised score only from 33.50 to 41.17 and from 35.83 to 40.50 (+7.67 and +4.67 points). Every in-pool answer ranks within the top 20, and swapping the ranker that picks the two attempts changes the score by −1.67 to +1.33 points. (iii) *Generation collapse*: a test input receives 14.0–14.4 candidates on average but only 7.3–7.6 distinct grids, and 48 and 47 of the test inputs without a correct candidate have candidates whose mean cell agreement with the answer exceeds 90 %. Across five full runs, the union of correctly answered test inputs grows from 51 to 62, and two runs of the same program with identical seeds answer 49 and 45. In both logged runs, 7–10 test inputs were lost in selection and 116–117 had no correct candidate: the score was bounded mainly by generation. We also document notebook diagnostics that misreport these quantities and list variants we did not measure: more samples, diversity-promoting search and other models.

1 Introduction

ARC-AGI-2 [6] continues the Abstraction and Reasoning Corpus [4]. Each task gives a few input–output grid pairs, and a solver must produce the output grid for one or more new test inputs. In the Kaggle competitions of the ARC Prize [5, 7], a solver runs offline under a fixed compute budget. It may submit two attempts per test input, and a test input counts as solved if either attempt matches the answer exactly. The winning entry of 2025, NVARC [12], combines a small language model fine-tuned on grids with test-time training [1] and augmentation-based candidate scoring in the style of Franzen et al. [8]. Its code and model are public, and our team built 34 Kaggle notebooks on it for ARC Prize 2026.

Improving such a pipeline requires knowing which stage limits it. A test input is lost either because no correct grid is ever generated or because a correct grid is generated but not among the two submitted. These two failure modes call for different work: better *selection* (rankers, voting,

*Kaggle ARC Prize 2026 (ARC-AGI-2 track).

verification) or better *generation* (search, sampling, model). Several of our notebook versions changed the selection logic (a "dual" selector, three alternative selectors). In this paper we measure how much headroom selection actually has.

We decompose pass@2 [3] into *pool coverage* (is a correct grid among the decoded candidates?) and *selection* (is it among the two submitted?). We measure both on two complete runs whose logs report, for every test input, where the answer ranks among the candidates. We complement this with all saved results of our NVARC notebooks (41 distinct results) to measure how much repeated runs add. Our contributions are:

- a decomposition of the NVARC pipeline's pass@2 score into pool coverage and selection on two full runs of the public ARC-AGI-2 evaluation set. The pool covers 55 and 56 of 172 test inputs, and perfect selection would add +7.67 and +4.67 task-normalised points (Section 4.1);
- a characterisation of the test inputs that never receive a correct candidate (68.0 % and 67.4 %): how close their candidates come to the answer, and how few distinct grids the search returns (Sections 4.2 and 4.3);
- the growth of the solved set over repeated full runs and of all our submissions together, including two runs of the same program with identical seeds that differ by 4 correct test inputs (Section 4.4);
- a list of measurement pitfalls in the notebooks' own diagnostics, with the corrected numbers (Section 5).

We report what we measured: one model with one search procedure, run in several code versions. We do not claim that no variant of the pipeline can do better. Section 7 lists the variants we did not test.

2 The NVARC pipeline as run in our notebooks

All runs in this study use the same model and the same solver code family. Settings below are taken from the solver source saved with run R2 (`arc_solver.py`, `arc_loader.py`, `arc_decoder.py`). Run R1 uses the same code except for one constant, noted below.

Our notebooks modify the released solver. Compared with our team's reproduction notebook of the 2025 release, the main differences are:

- the probmul ranker uses a per-grid baseline instead of a fixed baseline of 3;
- the submitted pair comes from the "dual" selector instead of the kgmon top 2;
- the per-task cap is 1600 s instead of 1200 s;
- augmentation seeds are derived from MD5 hashes instead of Python's randomised `hash()`;
- a placeholder grid is written when no output is produced.

Run A still uses the fixed baseline, the 1200 s cap and `hash()` seeds. The results below describe the pipeline as run in these notebooks. Figure 1 shows the architecture of our study. Panel (a) is the solver, with the stage at which each of our measurements applies. Panel (b) shows how the numbers are produced from the saved notebook outputs.

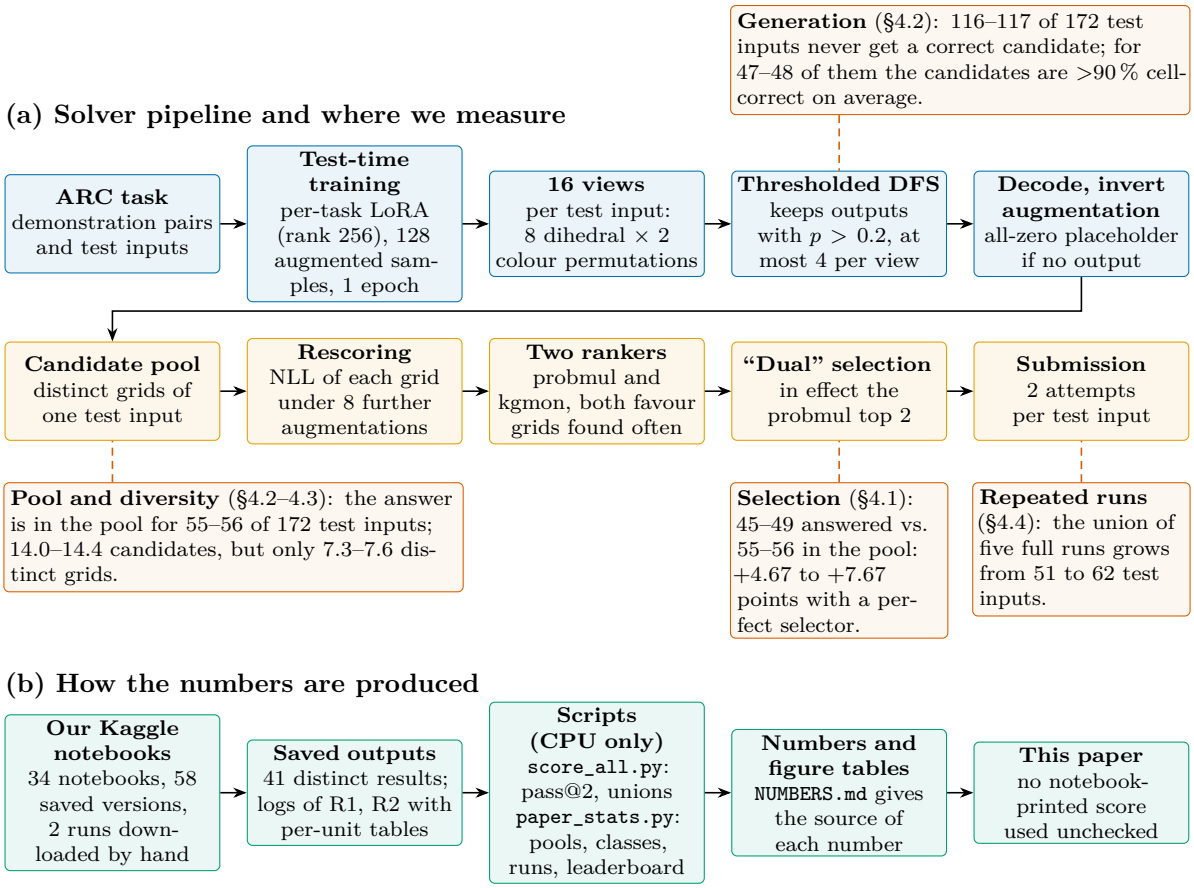


Figure 1: Architecture of the study. (a) The NVARC solver as run in our notebooks (blue: generation; orange: scoring and selection). Red boxes give the quantity measured at each stage, as ranges over the two logged runs R1 and R2 (five full runs for the union). (b) The measurement pipeline: every number is recomputed from saved outputs by two scripts that need no GPU.

Model and test-time training. The base model is `sorokin/qwen3_4b_grids15_sft139`, a Qwen3-4B [11] model fine-tuned on grid tasks, loaded in bfloat16 with Unsloth [13]. For every task, a fresh LoRA adapter [9] (rank 256, $\alpha = 32$, rank-stabilised scaling [10], applied to all attention and MLP projections, the embeddings and the output head) is trained for one epoch. The training data are augmented copies of the task's demonstration pairs: 8 dihedral transforms × 16 random colour permutations. This gives 128 training samples, and training runs 128 steps. Training uses learning rate $5 \cdot 10^{-5}$ with cosine decay, batch size 1 and a maximum sequence length of 8192 tokens.

Generation. Each test input is presented in up to 16 augmented views (8 dihedral transforms × 2 colour permutations). For each view, a depth-first search (`turbo_dfs`) keeps a partial output only while its cumulative negative log-likelihood stays below $-\ln 0.2 \approx 1.609$. A completed output is therefore one whose probability under the adapted model, given that view, exceeds 0.2. Let Y_v be the completed outputs of view v and C_u the multiset of candidates of test input u. Distinct completed outputs are prefix-free, so their probabilities sum to at most 1:

$$1 \;\geq\; \sum_{y \in Y_v} p_\theta(y \mid v) \;>\; 0.2\,|Y_v| \quad\Longrightarrow\quad |Y_v| \leq 4, \qquad |C_u| \leq 16 \cdot 4 = 64. \tag{1}$$

The code also caps the number of next tokens expanded per step: 10 in R2, 5 in R1, A and D. This cap never binds, because a token survives only if its conditional probability exceeds 0.2, and at most 4 tokens can. Each output is decoded into a grid and mapped back through the inverse augmentation. If the search returns no completed output for a test input, the pipeline writes an all-zero placeholder grid of the test-input shape.

Scoring. Every distinct grid y is re-scored by the adapted model under 8 further augmentations. Each occurrence g of y (from different views or search paths) carries its search NLL b_g and eight augmentation NLLs $a_{g,1..8}$. Two rankers order the distinct grids by

$$\text{probmul}(y) = \sum_{g \in G(y)} (\beta - b_g) + \frac{1}{|G(y)|} \sum_{g \in G(y)} \sum_{j=1}^{8} (\beta - a_{g,j}), \qquad \beta = \max_{g \in G(y)} b_g + 1, \tag{2}$$

$$\text{kgmon}(y) = |G(y)| - \frac{1}{|G(y)|} \sum_{g \in G(y)} \frac{1}{8} \sum_{j=1}^{8} a_{g,j}, \tag{3}$$

where $G(y)$ is the set of occurrences of y (functions `score_full_probmul_3` and `score_kgmon`). The augmentation NLLs are computed once per grid, so $a_{g,j}$ is the same for every occurrence of y. Both rankers reward grids found many times.

Selection. The submitted pair ("dual selection") is built from the top 3 grids of each ranker, de-duplicated with the probmul list first. Attempt 1 is the probmul top grid, and attempt 2 is the next grid in that pooled list. Both rankers order the same set of distinct grids. Attempt 2 is therefore the probmul rank-2 grid whenever there are two or more grids, and the kgmon ranking never changes the submitted pair. In both logged runs, the set of correctly answered test inputs equals the set whose answer probmul ranks 1 or 2 (Section 4.1). Algorithms 1 and 2 give the solver and the search as pseudocode, following the code.

Compute. Runs use a Kaggle machine with four NVIDIA L4 GPUs (one solver worker per GPU) and a 12-hour limit. A search call stops after 540 s. Before each batch of 4 views, generation for a task stops if the task has already used 1600 s in total, training included. A full run over all 120 evaluation tasks took 4.68–4.80 h (Table 1).

3 Data and measurement

Evaluation set and metric. We use the public ARC-AGI-2 evaluation set: 120 tasks with 172 test inputs. We call one test input a *unit*. A unit is correct if attempt 1 or attempt 2 equals the answer grid (pass@2). The *task-normalised score* gives each correct unit the weight $1/n_t$, where n_t is the number of test inputs of its task, and sums over units. Its maximum is 120. For unit u of task $t(u)$, with answer $y_u^\star$ and submitted attempts $\hat{y}_u^1, \hat{y}_u^2$,

$$c_u = \mathbf{1}\left[\, y_u^\star \in \{\hat{y}_u^1, \hat{y}_u^2\} \,\right], \qquad S = \sum_u \frac{c_u}{n_{t(u)}} \in [0, 120]. \tag{4}$$

The competition overview [7] words the leaderboard score as an average over test outputs. The leaderboard values in our records fit per-task averaging instead: all 18 distinct values are multiples of 100/720 to two decimals, as per-task averaging over 120 tasks with one to three test inputs produces, while no plain average over N test outputs ($100 \le N < 400$) fits them. We therefore read

Algorithm 1 The NVARC solver for one task, as run in our notebooks (code of R2). In the code, the batches of different test inputs of a task are interleaved.

Require: task t with demonstration pairs D_t and test inputs $x_1, \ldots, x_{n_t}$; base model θ_0; threshold $\tau = -\ln 0.2$

$\theta \leftarrow \theta_0$ plus a fresh LoRA adapter, trained one epoch on $\mathrm{Aug}(D_t)$ ▷ 8 dihedral × 16 colour perm.
for $i = 1, \ldots, n_t$ **do**
  $C_i \leftarrow \emptyset$ ▷ multiset of candidates $(y, b, \mathbf{a})$
  for each batch of 4 of the 16 views v of x_i **do** ▷ 8 dihedral × 2 colour perm.
    if task t has used more than 1600 s **then break**
    end if
    for each completed output $(\sigma, b) \in \textsc{ThresholdedDFS}(\theta, v(D_t, x_i), \tau)$ **do** ▷ Algorithm 2
      $g \leftarrow \mathrm{decode}(\sigma)$; **if** g is not a valid grid **then continue**
      $y \leftarrow v^{-1}(g)$ ▷ undo the augmentation
      if y has no augmentation scores yet **then**
        $\mathbf{a}_y \leftarrow \left(\mathrm{NLL}_\theta(w_j(y) \mid w_j(D_t, x_i))\right)_{j=1}^{8}$ ▷ 8 further augmentations w_j
      end if
      $C_i \leftarrow C_i \cup \{(y, b, \mathbf{a}_y)\}$
    end for
  end for
  if $C_i = \emptyset$ **then** $C_i \leftarrow \{(\mathbf{0}_{\mathrm{shape}(x_i)},\ 999,\ 999 \cdot \mathbf{1})\}$ ▷ placeholder
  end if
  $L_a \leftarrow$ distinct grids of C_i by decreasing probmul (2); $L_b \leftarrow$ the same by kgmon (3)
  $L \leftarrow \mathrm{dedup}(L_a[1..3] \,\|\, L_b[1..3])$
  submit $(L[1], L[2])$, or $(L[1], L[1])$ if $|L| = 1$ ▷ $L[2] = L_a[2]$ whenever $|L_a| \geq 2$
end for

the leaderboard as the task-normalised score expressed as a percentage. We report both units (of 172) and the task-normalised score (of 120). All scores are recomputed from the saved submission files against the public answers. For the two downloaded runs, no submission file was saved; we use the set of correct units printed by the decoder's built-in pass@2 benchmark, whose code compares the first two submitted grids with the answer.

Runs. Our team's Kaggle account holds 34 notebooks that use the NVARC solver, with 58 saved versions. Their outputs contain 39 distinct submission files (by SHA-256) from 20 of these notebooks. Two more runs (B and C in Table 1) were downloaded by hand as notebooks with their cell outputs. That makes 41 distinct results, most of them partial runs over a subset of tasks. Six runs trained and searched all 120 tasks (Table 1). Most of them are versions of the pipeline with code changes between them (fixes, selector variants, diagnostics). The exception is R1 and R2, versions 1 and 2 of one notebook. Their loader and decoder code is identical, and their solver differs only in the non-binding cap above. All 240 seed lines (one training and one evaluation seed per task) are identical. Yet their training losses differ (e.g. 0.018519 vs. 0.018989 on task `271d71e2`), and so do their results. They are two runs of one program, and we use them to measure run-to-run variation (Section 4.4). These two runs end with analysis cells that print, for every unit, the rank of the answer among the decoded candidates under both rankers, the number of candidates and distinct grids, and the mean cell agreement of the candidates with the answer. The candidate grids themselves were not saved. We parse these tables from the run logs and use our own counts. The "candidate

Algorithm 2 Thresholded depth-first search (`turbo_dfs`) for one view. In the code, the views of a batch are searched together, and a call stops after 540 s. K is 10 in R2 and 5 in R1, A and D; it never binds (Section 2).

function THRESHOLDEDDFS(θ, prompt h, τ)
  return EXPAND(h, 0, $m_{\max}$) ▷ each returned (σ, s) has $p_\theta(\sigma \mid h) = e^{-s} > e^{-\tau} = 0.2$
end function
function EXPAND(h, s, m) ▷ s: NLL of the partial output, m: tokens left
  $O \leftarrow \emptyset$
  for each grid token w with $s_w = s - \log p_\theta(w \mid h) < \tau$, by increasing s_w **do**
    if $w =$ EOS **then**
      $O \leftarrow O \cup \{(\text{EOS}, s_w)\}$
    else if $m > 1$ and w is among the K best non-EOS tokens **then**
      $O \leftarrow O \cup \{(w \| \sigma, s') : (\sigma, s') \in \text{EXPAND}(h \| w, s_w, m-1)\}$
    end if
  end for
  return O
end function

pool" P_u of a unit is the set of distinct grids the run decoded for it (the distinct elements of C_u). A unit is *in the pool* if one of them equals the answer. *Oracle selection* submits the answer for every in-pool unit; its score is the pool coverage O. With r_u the probmul rank of $y_u^\star$ in P_u ($r_u = \infty$ if it is absent),

$$O = \sum_u \frac{\mathbf{1}[\, y_u^\star \in P_u \,]}{n_{t(u)}}, \qquad H = O - S, \qquad C(k) = \big|\{\, u : r_u \le k \,\}\big|. \tag{5}$$

H is the selection headroom and $C(k)$ the top-k coverage. The submitted grids belong to P_u, so $S \le O$, and every unit falls into exactly one of three groups: correct, in the pool but not submitted, or not in the pool. Algorithm 3 gives the decomposition as pseudocode.

Leaderboard records. Public-leaderboard scores (computed on hidden tasks) come from our team's submission list: 63 scored submissions from 19 notebooks. Each submission is a fresh run of the notebook. Leaderboard scores are percentages, not points out of 120.

Reproducibility. `scripts/score_all.py` re-scores every submission and computes the unions. `scripts/paper_stats.py` computes the pool, generation, diversity, union and leaderboard statistics and the figure data from the saved logs and submissions. Neither script needs a GPU. The few numbers quoted directly from a run log are marked as printed by the run. `paper/NUMBERS.md` lists the source of every number.

4 Results

4.1 Selection headroom is small

Table 2 decomposes the two logged runs. In R2, the submission answers 45 units (33.50). A correct grid is present for 55 units, so oracle selection would reach 41.17: a headroom of +7.67 points (10 units). In R1, the submission answers 49 units (35.83) and the pool covers 56 (40.50): a headroom of +4.67 points (7 units). The rest of the gap to a perfect score is not a selection problem: 117 units in R2 and 116 in R1 have no correct candidate at all.

Algorithm 3 Pool decomposition of one logged run (`scripts/paper_stats.py`).

Require: per-unit table of the run (r_u, $|C_u|$, $|P_u|$, a_u, placeholder flag); the run's submission; the answers $y^\star$

for each of the 172 units u **do**
  if $c_u = 1$ **then**
    class(u) $\leftarrow$ correct ▷ Equation (4)
  else if $r_u < \infty$ **then**
    class(u) $\leftarrow$ in the pool, not submitted
  else if u has only the placeholder grid **then**
    class(u) $\leftarrow$ no completed output
  else if $a_u > 90$ **then**
    class(u) $\leftarrow$ close ▷ Equation (6)
  else if $a_u > 50$ **then**
    class(u) $\leftarrow$ partial
  else
    class(u) $\leftarrow$ far
  end if
end for
compute S, O, H and $C(k)$ (Equations (4) and (5)); assert $\{u : c_u = 1\} \subseteq \{u : r_u < \infty\}$
report whether $\{u : c_u = 1\} = \{u : r_u \leq 2\}$, and the duplicate ratios d_u (Equation (7))
over all full runs: union growth U_m (Equation (8)) for every order of the runs

Figure 2 shows the top-k coverage $C(k)$ of Equation (5), i.e. where the answer sits in the probmul ranking when it is present. It is ranked first for 36 (R2) and 39 (R1) units, within the top 3 for 51 and 52, and within the top 10 for 55 and 55. Every in-pool answer is ranked within the top 20. A third attempt, if the rules allowed one, would add 6 and 3 units. The two rankers are close to redundant. They put the answer at the same rank for 48 of 55 (R2) and 51 of 56 (R1) in-pool units; the Pearson correlations of the two ranks are 0.961 and 0.991. Changing the selector moves the score little. Submitting the kgmon top 2 instead of the probmul top 2 would answer 47 units (34.83, +1.33) in R2 and 48 units (34.17, −1.67) in R1; the decoders' own benchmarks print the same values. Run D evaluates three selectors on one candidate pool, scoring the two grids each would submit. Its log reports 33.67, 33.67 and 34.67 (`CURRENT`, `IMBUE_EXACT`, `IMBUE_QWEN_DIVERSE`; as printed by the run). The first value matches our re-scoring of D's submission.

4.2 The limit is generation

In R2, 117 of 172 units (68.0 %) never receive a correct candidate; in R1, 116 (67.4 %). Figure 3 splits these units by how close the candidates come to the answer. Closeness is measured by the mean cell agreement with the answer over all candidates of the unit, where a candidate whose shape differs from the answer counts as 0:

$$a_u = \frac{100}{|C_u|} \sum_{c \in C_u} \mathbf{1}\big[\mathrm{shape}(c) = \mathrm{shape}(y^\star_u)\big] \cdot \frac{1}{|y^\star_u|} \sum_{(i,j)} \mathbf{1}\big[c_{ij} = y^\star_{u,ij}\big], \tag{6}$$

with $|y^\star_u|$ the number of cells of the answer (notebook cell 30 of R2). We use the cut points of the runs' own analysis: above 90 % is *close*, 50–90 % *partial*, otherwise *far*.

- **Close (48 in R2, 47 in R1).** The mean agreement over all candidates exceeds 90 %, yet no

Table 1: The six full runs (all 120 tasks trained and searched). Scores are recomputed from the submission files. For B and C († downloaded by hand as notebooks with outputs; no submission file saved), they are recomputed from the correct-unit set printed by the decoder's pass@2 benchmark. R1 and R2 log per-unit pool statistics.

label	notebook, version	wall (h)	units (/172)	task-norm (/120)
A	`arc-2026d`, v4	4.68	51	37.67
R1	`arc-2026d-with-codex`, v1	4.77	49	35.83
R2	`arc-2026d-with-codex`, v2	4.75	45	33.50
B	`fork-of-arc-2026d-32-22lb-updated-v3`†	4.69	51	37.83
C	`arc-2026d-with-codex`, "v25"†	4.75	51	37.67
D	`experiment-with-arc-agi-2`, v10	4.80	47	33.67

Table 2: Pool coverage and selection in the two logged runs (units of 172; task-normalised score of 120 in parentheses).

	R1	R2
submitted pair correct (pass@2)	49 (35.83)	45 (33.50)
answer in candidate pool (oracle selection)	56 (40.50)	55 (41.17)
selection headroom	+7 (+4.67)	+10 (+7.67)
no correct candidate	116 (67.4 %)	117 (68.0 %)
answer at probmul rank 1	39	36
answer at probmul rank ≤ 2 (= submitted pair)	49	45
answer at probmul rank ≤ 3	52	51
answer at probmul rank ≤ 5	54	54
answer at probmul rank ≤ 10	55	55
answer at probmul rank ≤ 20	56	55
same rank under probmul and kgmon	51 of 56	48 of 55
kgmon top 2 submitted instead	48 (34.17)	47 (34.83)

candidate is exact. Because a candidate of another shape counts as 0, most of these candidates have the right shape.

- **Partial (33, 32)** and **far (22, 22).**
- **No completed output (14, 15).** The search returned no output above the probability threshold, the log records `FALLBACK_SAVE ... NOT_PROCESSED`, and an all-zero placeholder was submitted. These units belong to 11 tasks in each run. All of them finished normally (in 182–802 s in R2 and 134–873 s in R1). Neither run logged a task time-out or an exception. The longest search call, including scoring, took 246.9 s in R2 and 227.1 s in R1, below the 540 s cap.

Units whose answer is in the pool but was not submitted (10 and 7) are the only losses that a better selector could recover.

A single task shows what these near misses look like. For task `142ca369`, R2's repair analysis prints the closest candidates for both test inputs (as printed by the run). They agree with the answer on 72.84 % and 93.16 % of cells, and at every one of the 88 and 26 differing cells the candidate predicts background (colour 0) where the answer has a colour. In R1, the closest candidate for

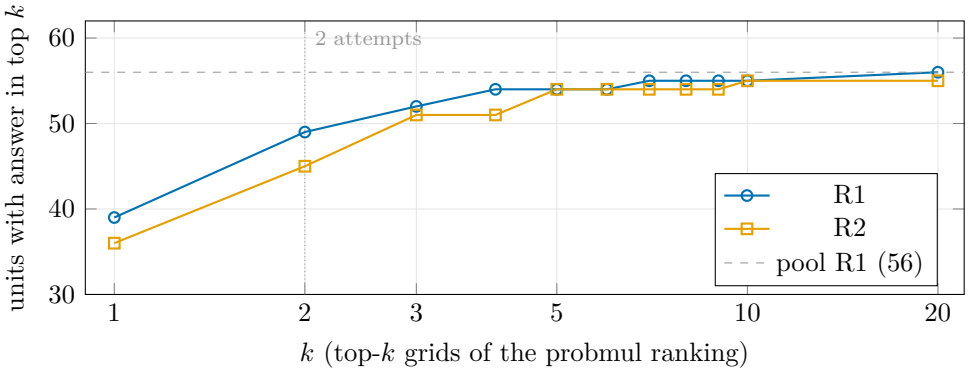


Figure 2: Top-k coverage of the answer in the two logged runs. The submitted pair equals the probmul top 2 ($k = 2$); beyond $k = 10$ almost nothing remains to select (the pool of R2 has 55 units, that of R1 56).

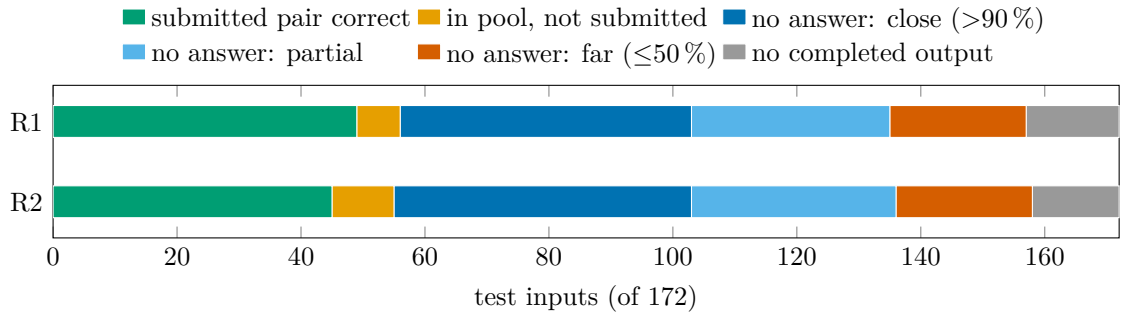


Figure 3: Where each test input ends in the two logged runs. Only the orange segment can be recovered by selection; the other unsolved segments have no correct candidate.

the first test input agrees on 95.68 %; its 14 wrong cells have colour 2 where the answer has 6. No ranking of such candidates can produce the answer.

4.3 The candidate pools have few distinct grids

A unit receives 14.0 candidates on average in R2 and 14.4 in R1, but only 7.3 and 7.6 distinct grids. The duplicate ratio of a unit is

$$d_u = 1 - \frac{|P_u|}{|C_u|}, \tag{7}$$

and its mean over the 172 units is 0.378 in R2 and 0.381 in R1. Excluding the placeholder units, the averages are 15.20 candidates and 7.85 distinct grids in R2, and 15.64 and 8.19 in R1. The largest pool has 27 candidates, well below the bound of 64 from Section 2. Figure 4 shows the distribution of distinct grids per unit. Thirty-three units in R2 and 26 in R1 have at most 3 distinct grids, and for 19 and 14 of these, none of the grids is the answer. For example, for `5961cc34` (test input 0), all 16 candidates are the same wrong grid in both runs, with 95.9 % of cells right.

This is consistent with how candidates are generated. The search is deterministic and keeps only outputs with probability above 0.2. The views differ only by symmetry and colour relabelling, and they often lead the adapted model to the same grid. The pool therefore holds, in effect, a few of the model's highest-probability outputs. In units such as `5961cc34`, all views produce the same

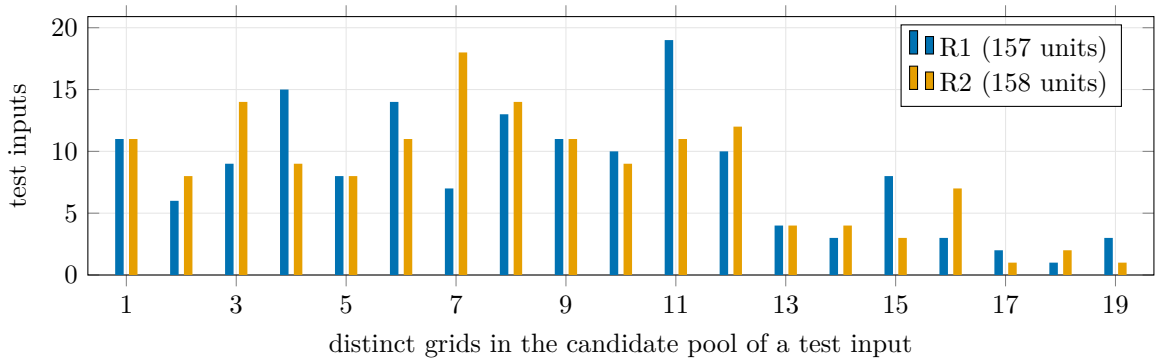


Figure 4: Distinct grids per test input, excluding the units with only a placeholder grid. Each unit is searched in up to 16 views.

wrong grid. In units with several distinct grids, the rankers favour the grids found most often. The search has no term that rewards diverse outputs [cf. 14], and the pipeline does not draw stochastic samples. We did not measure whether either would add correct grids (Section 7).

4.4 Repeated runs re-solve the same test inputs

The six full runs score 33.50–37.83 (45–51 units; Table 1). With $R_j = \{u : c_u = 1 \text{ in run } j\}$, the union after m runs is

$$U_m = \Big| \bigcup_{j=1}^{m} R_j \Big|. \tag{8}$$

Figure 5 shows U_m over five of the full runs (A, R1, R2, B, C, in this order): 51, 57, 58, 61 and 62 units, or 37.67, 42.00, 42.50, 44.00 and 45.00 points. Over all 120 orders of these five runs, the union after two runs lies between 52 and 59 units, and it reaches 62 after all five. The sixth full run, D, adds no new unit. Thirty-seven units (27.50 points) are correct in all five runs; of the 62 units in the union, 4 are correct in only one run. Any two of these five runs share 41–45 correct units out of a union of 52–59.

R1 and R2 isolate run-to-run variation, because they run the same program with the same seeds (Section 3). They answer 49 and 45 units (35.83 and 33.50), share 42, and together answer 52. Their pools hold 56 and 55 units, share 50, and together hold 61 (44.50). Repeated runs therefore do add correct candidates. The union of the five full runs (45.00) and the union of the two pools (44.50) both exceed the oracle score of either single run (41.17, 40.50). The additions are small, however. The union of all 41 distinct results, including partial runs, is 64 units (46.50 points, 38.75 %), against 51 units (37.83) for the best single run. Submissions and pools together cover 70 units (50.83 points, 42.36 %). For the remaining 102 units, no submission and neither logged pool contains the answer.

On the public leaderboard (hidden tasks), the best NVARC submission of our team scored 32.64 %. Ten submissions of that same notebook version, each a fresh run, scored between 27.36 % and 32.64 % (mean 29.36 %). A single leaderboard score of this pipeline therefore says little about a code change of a few points.

5 Pitfalls in the notebooks' own diagnostics

The analysed notebooks print their own diagnostics. Several of them would lead to the opposite conclusion if quoted as printed. We list them because they are easy to repeat in any pipeline of this

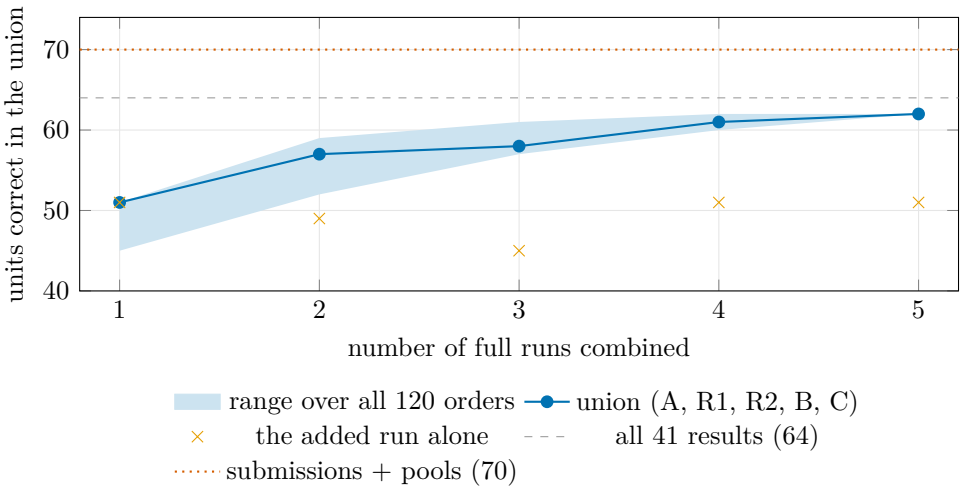


Figure 5: Growth of the set of correctly answered test inputs as full runs are combined.

kind. Every number in this paper is recomputed, or quoted from a run log and marked as printed by the run.

1. **Generation losses reported as ranking losses.** A survival table in R2 prints "Lost at ranking stage: 117 (68.0 %)". These 117 units have no correct candidate at all; the loss happens in generation.

2. **Inverted comparisons.** A score audit in R2 labels the answer as having the "better" search and augmentation score when its NLLs are higher (worse), e.g. augmentation mean 12.78 vs. 4.18 for the rank-1 grid.

3. **A losing variant reported as the winner, from a test of a different ranker.** The "scoring optimization" experiment in R2 prints "WINNER" and a recommendation for a variant that is 4.67 points below the baseline. All seven variants lost (between −4.67 and −9.33, as printed). The variants are not re-weightings of the shipped ranker. The experiment scores every occurrence of a grid on its own, which drops the occurrence count that both shipped rankers reward and lets one grid fill both attempts. With a per-occurrence baseline, the search term is also constant, so four of the variants are one and the same ranking and print the same score (26.50).

4. **Placeholders reported as crashes.** Units with only the all-zero placeholder are labelled "Worker crashed or timed out". In the logs, these tasks finished normally and the search returned no completed output (Section 4.2). A second label, "Pruned by max_score", describes the outcome correctly, but the notebook derives it from the placeholder's dummy score of 999, not from the search.

5. **Simulations that cannot see what they claim to measure.** A threshold sweep only re-filters candidates that already passed the real threshold, so it cannot show pruned answers. A "beam width" sweep is top-k of the existing pool. A meta-selector simulation reports 0.00 against a 33.50 baseline.

6. **Headroom counted from the wrong baseline.** The top-k experiment prints that selection "can recover at most 18 additional puzzles" (R2; 15 in R1), counting from top 1. Two attempts

are already submitted, so the gain to top 5 is 9 units (R2) and 5 (R1).

7. **Scores over the wrong denominator.** Run B prints "37.8/111", i.e. over the 111 tasks with any decoded output. The score is 37.83 of 120 (51 of 172 units).

8. **Pool coverage reported as solved.** Run B's summary "47/120 puzzles solved" counts a task if any candidate anywhere is correct (an oracle). A later cell's "34/120 puzzles solved by dual selection" counts only answers at rank 1 and ignores attempt 2. Both differ from the decoder's own pass@2 benchmark, whose correct-unit set we use for B (Table 1).

9. **Methods that do not match the code.** Run B's description says the dual selector takes the top grid of each of the two rankers. As shown in Section 2, attempt 2 is the probmul rank-2 grid whenever one exists, and kgmon never changes the submitted pair. Run B prints the same score, 37.8, for probmul alone and for the dual selector.

6 Related work

Test-time training on augmented demonstrations [1] and augmentation-based scoring of candidates [8] were key ingredients of the strongest open ARC Prize 2024 entries [5]. NVARC [12] carries them into the ARC-AGI-2 setting [6]. Outside ARC, repeated sampling studies [2] separate coverage (does any sample solve the problem?) from selection (can the solver pick it?), the same split we measure here. Diverse decoding [14] targets the low diversity we observe in the candidate pools.

7 Limitations

- **One model and one search procedure.** All runs use the same 4B model, the same test-time training recipe and the same thresholded search, as modified in our notebooks (Section 2). The results describe this pipeline as it was run. They do not show that no NVARC variant can exceed these numbers.

- **Generation-side variants were not measured.** We did not test more views or samples, a lower probability threshold, stochastic sampling, a diversity term in the search, longer test-time training or a larger model. Any of these could change the pool. Two runs of the same program already reach a pool union of 61 units against 55 and 56 for each run alone, so more sampling could add correct grids; how many is open.

- **Two logged pools.** Pool-level statistics come from two runs of one program. The other four full runs provide submission-level data only. The two runs differ by 4 correct units, so single-run numbers carry run-to-run noise of that order.

- **Quoted diagnostics.** Run D's three-selector comparison and the 142ca369 cell counts are quoted from the run logs; the candidate grids were not saved, so we could not recompute them. D's baseline value matches our re-scoring. The rank correlations and duplicate ratios we recomputed from the per-unit tables.

- **Public evaluation set.** All pool and union statistics are on the public evaluation set; the leaderboard uses hidden tasks, for which we only have scores.

- **Coarse closeness measure.** Mean cell agreement over all candidates ignores which cells are wrong and penalises shape errors fully. It does not measure how hard the remaining error is to fix.

8 Conclusion

On the public ARC-AGI-2 evaluation set, the NVARC pipeline as we ran it is capped by its candidate pool in both runs whose pools we logged. Perfect selection would add only +4.67 to +7.67 task-normalised points. About two thirds of the test inputs (67.4–68.0 %) never receive a correct candidate, often while the candidates are more than 90 % correct, and the pool holds few distinct grids. Repeated runs mostly re-solve the same test inputs, and resubmissions of the same notebook version vary by up to 5.28 percentage points on the leaderboard. For this pipeline, work on selection has a small, measurable ceiling. Generation-side changes are where the remaining headroom would have to come from, and we did not measure them.

Code and data. The scripts, run logs, submissions and extracted tables are available at `https://github.com/Foysal348/ARC-AGI-2` (folder `papers/1 - NVARC Solution 2025 Limitation`).

References

[1] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

[2] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[5] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical Report. *arXiv preprint arXiv:2412.04604*, 2024.

[6] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[7] François Chollet, Mike Knoop, Greg Kamradt, Walter Reade, María Cruz, and Addison Howard. ARC Prize 2026 - ARC-AGI-2. `https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2`, 2026. Kaggle.

[8] Daniel Franzen, Jan Disselhoff, and David Hartmann. Product of experts with LLMs: Boosting performance on ARC is a matter of perspective. *arXiv preprint arXiv:2505.07859*, 2025.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[10] Damjan Kalajdzievski. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*, 2023.

[11] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[12] Ivan Sorokin and Jean-François Puget. NVARC solution to ARC-AGI-2 2025. `https://github.com/1ytic/NVARC`, 2025. Write-up `nvarc_2025.pdf` in the repository. Model: `https://www.kaggle.com/models/sorokin/qwen3_4b_grids15_sft139`.

[13] Unsloth AI. Unsloth. `https://github.com/unslothai/unsloth`, 2024.

[14] Ashwin K. Vijayakumar, Michael Cogswell, Ramprasaath R. Selvaraju, Qing Sun, Stefan Lee, David Crandall, and Dhruv Batra. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424*, 2016.